UNIT 5 · TOPIC 5.5

Protecting Applications

Assignment, Answer Key

Teacher answer key, 30 points total.

Practice Questions

5.5.a · Input validation and sanitization

1. Which validation strategy defines exactly what valid input looks like and rejects everything else, rather than trying to enumerate every bad pattern? (1 pt)

○ A. Denylist (blocklist)

● **B. Allowlist (whitelist)**

○ C. Output encoding

○ D. Canonicalization

Why: Allowlisting defines the known-good shape of input and rejects anything that doesn't match; it does not need to anticipate every attack variant.

2. Select ALL statements that correctly describe why client-side (JavaScript) validation is not a security control. (3 pts)

■ **A. A browser is entirely under the attacker's control, so JavaScript checks can be disabled or bypassed**

■ **B. An attacker can send a crafted request directly to the server with a tool like curl, skipping the browser entirely**

■ **C. Client-side validation must still be re-enforced on the server for any security-relevant check**

□ D. Client-side validation is cryptographically enforced and cannot be bypassed

Why: Client-side validation only improves UX; because the client is attacker-controlled, every security-relevant check must be re-verified server-side.

3. An application validates a raw, un-decoded path string before checking it for '../' sequences. Why is this a mistake? (1 pt)

○ A. It has no effect on security

○ B. It makes the application faster

● **C. An attacker can URL-encode the traversal sequence (e.g. ..%2F) so it slips past the raw-string check, then gets decoded later, this is a canonicalization bypass**

○ D. Only client-side code is affected

Why: Validating before canonicalizing (decoding/normalizing) lets encoded or obfuscated payloads bypass the filter; the correct order is decode/normalize first, then validate.

4. Name the security concept that transforms data at the point it is inserted into a new context (HTML, SQL, shell) so special characters are treated as literal data, not code. (1 pt)

Acceptable answers: output encoding; output escaping; encoding; escaping; context-specific encoding

Why: Output encoding/escaping is applied at each output context, since the correct transformation differs between HTML, SQL, shell, and JSON.

5. SQL injection, cross-site scripting (XSS), and command injection share what root cause? (1 pt)

○ A. They all require physical access to the server

● **B. The application trusted that input was 'just data' when a downstream interpreter treated part of it as code or control syntax**

○ C. They only affect encrypted connections

○ D. They are unrelated vulnerability classes with no common cause

Why: In all three, unvalidated input reaches an interpreter (SQL engine, browser, shell) that treats some of its characters as executable syntax rather than literal data.

5.5.b · Secure coding practices

6. Which practice eliminates SQL injection risk regardless of what input validation is or isn't done upstream? (1 pt)

● **A. Parameterized queries**

○ B. Verbose error messages

○ C. Running the app as root

○ D. Hardcoding secrets in source

Why: Parameterized queries bind user input as data, never as SQL syntax, closing off injection independent of validation.

7. Select ALL reasons secrets (API keys, passwords) should NOT be hardcoded in source code. (3 pts)

■ **A. They remain visible in version control history forever, even if later removed**

■ **B. Rotating a compromised secret requires a code change and redeployment**

□ C. They are automatically encrypted by the version control system

■ **D. Anyone with repository read access can see them**

Why: Hardcoded secrets persist in history, require a code change to rotate, and are visible to anyone with repo access; version control does not encrypt file contents.

8. An authentication check throws an unexpected exception. Secure coding says the application should: (1 pt)

○ A. Grant access anyway so the user isn't blocked

○ B. Ignore the exception silently and continue

○ C. Log the user in as admin

● **D. Deny access by default (fail securely/fail closed)**

Why: Failing securely means an unexpected error must never accidentally grant access; deny by default.

9. Name the principle that says application processes and database connections should run with only the minimum access they need to function. (1 pt)

Acceptable answers: least privilege; principle of least privilege

Why: Least privilege applies to running code and service accounts, not just human user accounts.

10. Why is pinning and patching third-party dependency versions considered a secure-coding practice, not just a maintenance task? (1 pt)

● **A. A vulnerability in any dependency becomes a vulnerability in the whole application, so unpatched dependencies are unpatched attack surface**

○ B. It has no security relevance

○ C. It only affects build speed

○ D. Dependencies cannot contain vulnerabilities

Why: Modern apps are built on many third-party libraries; an unpatched vulnerable dependency is a real vulnerability in the shipped application.

5.5.c · Application security testing (SAST/DAST)

11. Which testing approach reads an application's source code without executing it? (1 pt)

- ○ A. DAST
- ○ B. Fuzzing only
- ○ C. Penetration testing
- **● D. SAST**

Why: SAST (static application security testing) parses source code directly and never runs the program.

12. Select ALL statements that correctly describe the tradeoffs between SAST and DAST. (3 pts)

- **■ A. SAST can run before the code is even merged, giving very early feedback**
- **■ B. DAST requires a running, reachable instance of the application**
- **■ C. SAST tends to produce more false positives because it cannot always confirm attacker-controlled input reaches a flagged line**
- □ D. DAST can see every code path in the application, including rarely-used admin features

Why: SAST runs early and over-flags because it lacks runtime context; DAST needs a live instance and only sees the paths it actually exercises, not every code path.

13. A dynamic scanner sends a crafted request to a live login form and observes that it can bypass authentication. What kind of finding is this? (1 pt)

- ○ A. SAST-only, since it's about code logic
- **● B. DAST, since it required interacting with the running application**
- ○ C. Neither, since automated tools cannot find this
- ○ D. A compiler warning

Why: DAST tests the running application from the outside, exactly the way this scenario describes.

14. Name the kind of security testing that is performed by a skilled human who chains findings together and reasons about business logic flaws automated SAST/DAST tools cannot recognize. (1 pt)

Acceptable answers: penetration testing; pen testing; pentest; pentesting

Why: Penetration testing is human-driven and can reason about business logic in ways automated scanners cannot.

15. Why do mature CI/CD pipelines typically run BOTH SAST and DAST rather than picking just one? (1 pt)

- ○ A. SAST and DAST are the same tool with different names
- ○ B. Running both is required by law in every jurisdiction
- ○ C. DAST always finds a superset of what SAST finds
- **● D. They find largely non-overlapping vulnerability classes, so together they give broader coverage**

Why: SAST sees the code but not runtime behavior; DAST sees runtime behavior but not the code, they complement rather than duplicate each other.

Coding Assignments (Python 3)

5.5.a · Input validation and sanitization

3 pts · auto-graded on the site

Coding problem, 5.5.a: Field validator (allowlist + injection detector)

Skill: Mitigate Risk · **Maps to:** 5.5.A · **Language:** Python 3 (pytest)

Problem

Build the server-side validation gate described in the lesson: given a batch of submitted form field values, classify each as SAFE or BLOCKED using **allowlist rules per field type** plus detection of known **injection meta-characters/keywords**, the same two checks a real input-validation layer runs before any value reaches a database query, HTML page, or shell.

Function contract

Implement `solve(submissions: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`submissions`: `list[dict]`, each `{"field": str, "type": str, "value": str}`. `type` is one of `"zip"`, `"email"`, `"quantity"`, `"username"`.

Output

`list[str]` of the `value`s that are BLOCKED, in the same order they appeared in the input.

Allowlist rules (per `type`)

- `zip`: exactly 5 digits.
- `email`: `local@domain.tld` shape.
- `quantity`: integer string 1-99.
- `username`: 3-20 letters/digits/underscore.

A value is also BLOCKED (even if it matches the allowlist shape) if it contains a known injection signature: `--`, `;`, `<script`, `UNION SELECT`, `OR 1=1`, `' OR '`, `DROP TABLE`, `../`, `%3B`, `%2F` (case-insensitive).

Examples

```
[{"field":"zip","type":"zip","value":"94107"}]                    ->  []
[{"field":"zip","type":"zip","value":"9410"}]                     ->  ["9410"]
[{"field":"u","type":"username","value":"admin'--"}]               ->  ["admin'--"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 5.5.a. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
import re
from typing import Dict, List

# Allowlist patterns per field type (server-side rules — see atom.md).
_ALLOW = {
    "zip": re.compile(r"^\d{5}$"),
    "email": re.compile(r"^[A-Za-z0-9._+-]+@[A-Za-z0-9-]+\.[A-Za-z0-9.-]+$"),
    "quantity": re.compile(r"^(?:[1-9][0-9]?)$"),  # 1-99
    "username": re.compile(r"^[A-Za-z0-9_]{3,20}$"),
```

```
}

# Known injection meta-character / keyword signatures, checked regardless of type.
_INJECTION = re.compile(
    r"(--|;|<script|</script|UNION\s+SELECT|OR\s+1\s*=\s*1|'\s*OR\s*'|\bDROP\s+TABLE\b|"
    r"\.\./|%3B|%2F)",
    re.IGNORECASE,
)


def solve(submissions: List[Dict[str, str]]) -> List[str]:
    """Classify each submission; return the `value`s that are BLOCKED, in input order.

    A submission is BLOCKED if its value fails the type's allowlist pattern, or if it
    contains a known injection meta-character/keyword signature (checked independent of type).
    """
    blocked = []
    for sub in submissions:
        field_type = sub["type"]
        value = sub["value"]
        pattern = _ALLOW.get(field_type)
        allow_ok = bool(pattern and pattern.match(value))
        has_injection = bool(_INJECTION.search(value))
        if not allow_ok or has_injection:
            blocked.append(value)
    return blocked
```

5.5.b · Secure coding practices

3 pts · auto-graded on the site

Coding problem, 5.5.b: Secure-coding configuration auditor

Skill: Mitigate Risk · **Maps to:** 5.5.A, 5.5.B · **Language:** Python 3 (pytest)

Problem

Build a lightweight policy-as-code auditor: given an application's configuration, flag every secure-coding practice it violates, exactly the checks a code reviewer or CI gate would run.

Function contract

Implement `solve(config: dict) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`config`: `dict` of settings. Any key may be missing.

Output

Sorted `list[str]` of violated rule names.

Rule set

condition	violation
`debug_mode is True`	`"debug-mode-enabled"`
`db_query_style == "concatenated"`	`"non-parameterized-query"`
`secrets_in_code is True`	`"hardcoded-secret"`
`error_verbosity == "verbose"`	`"verbose-error-messages"`
`dependencies_pinned is False`	`"unpinned-dependencies"`
`runs_as == "root"`	`"excessive-privilege"`
`code_review_required is False`	`"no-code-review"`

Examples

```
{"debug_mode": True}                          ->  ["debug-mode-enabled"]
{"db_query_style": "parameterized"}           ->  []
{"runs_as": "root", "secrets_in_code": True} ->  ["excessive-privilege", "hardcoded-secret"]
```

Constraints

- Deterministic, offline, standard library only.
- A missing key never triggers a violation.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 5.5.b. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from typing import Dict, List


def solve(config: Dict[str, object]) -> List[str]:
    """Audit an application configuration against the secure-coding rule set.

    Returns a sorted list of violated rule names (only rules that fail appear).
    """
    violations = []

    if config.get("debug_mode") is True:
        violations.append("debug-mode-enabled")

    if config.get("db_query_style") == "concatenated":
        violations.append("non-parameterized-query")

    if config.get("secrets_in_code") is True:
        violations.append("hardcoded-secret")

    if config.get("error_verbosity") == "verbose":
        violations.append("verbose-error-messages")

    if config.get("dependencies_pinned") is False:
        violations.append("unpinned-dependencies")

    if config.get("runs_as") == "root":
        violations.append("excessive-privilege")

    if config.get("code_review_required") is False:
        violations.append("no-code-review")

    return sorted(violations)
```

5.5.c · Application security testing (SAST/DAST)

3 pts · auto-graded on the site

Coding problem, 5.5.c: Static-analysis scanner (SAST)

Skill: Detect Attacks, Mitigate Risk · **Maps to:** 5.5.B, 5.5.C · **Language:** Python 3 (pytest)

Problem

Build a miniature SAST rule engine. Given the source lines of a file (as a list of strings, one per line, 1-indexed), scan each line for known dangerous patterns and return a list of findings, exactly what a real static-analysis tool's rule engine does before a human triages the results.

Detect these categories:

- `"sql-injection-risk"`, a SQL keyword (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) combined with string concatenation (`+`) or an f-string/`.format(`/`%` on the same line (i.e., the query is being built dynamically instead of parameterized).
- `"eval-use"`, the line calls `eval(` or `exec(`.
- `"hardcoded-secret"`, the line assigns a literal string to a variable whose name contains `password`, `secret`, `api_key`, or `token` (case-insensitive), e.g. `password = "hunter2"`. An assignment that reads from an environment variable or config object (no literal string on the right-hand side) is NOT a finding.

Function contract

Implement `solve(lines: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `lines`: `list[str]`, source code lines, in file order. Line numbers are 1-indexed (the first element of the list is line 1).

Output

A `list[dict]`, each `{"line": int, "category": str}`, **sorted by line number ascending**, then by category name ascending for findings on the same line. If a line matches more than one category, emit one finding per matching category. If no lines match, return `[]`.

Examples

```
lines = [
  'query = "SELECT * FROM users WHERE name = \'" + user_input + "\'"',
  'password = "hunter2"',
  'eval(user_expr)',
]
solve(lines) -> [
  {"line": 1, "category": "sql-injection-risk"},
  {"line": 2, "category": "hardcoded-secret"},
  {"line": 3, "category": "eval-use"},
]
```

```
lines = ['password = os.environ["DB_PASSWORD"]', 'db.query(stmt, params)']
solve(lines) -> []
```

Constraints

- Deterministic, offline, standard library only.
- Findings are sorted by `line`, then `category` (both ascending).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 5.5.c. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
import re
from typing import Dict, List

_SQL_KEYWORD = re.compile(r"\b(SELECT|INSERT|UPDATE|DELETE)\b", re.IGNORECASE)
_DYNAMIC_BUILD = re.compile(r"\+|f[\"']|\.format\(|%\s*[\"']|[\"']\s*%")
_EVAL_USE = re.compile(r"\b(eval|exec)\s*\(")
_SECRET_ASSIGN = re.compile(
    r"\b\w*(password|secret|api_key|token)\w*\s*=\s*[\"']", re.IGNORECASE
)


def solve(lines: List[str]) -> List[Dict[str, object]]:
    """Scan source lines for known dangerous patterns (miniature SAST rule engine).

    Returns findings sorted by line ascending, then category ascending; a line may
    contribute more than one finding.
    """
    findings = []
    for i, line in enumerate(lines, start=1):
        cats = set()
        if _SQL_KEYWORD.search(line) and _DYNAMIC_BUILD.search(line):
            cats.add("sql-injection-risk")
        if _EVAL_USE.search(line):
            cats.add("eval-use")
        if _SECRET_ASSIGN.search(line):
            cats.add("hardcoded-secret")
        for cat in cats:
            findings.append({"line": i, "category": cat})
    findings.sort(key=lambda f: (f["line"], f["category"]))
    return findings
```